

zh3r0 2020 Crypto - Hastads Message

deuterium — 2020-06-18

Canonical HTML: [https://deut-erium.github.io/WriteUps/2020/zh3r0/crypto/Hastads message/2020-06-18-zh3r0-2020-Hastads-Message.html](https://deut-erium.github.io/WriteUps/2020/zh3r0/crypto/Hastads%20message/2020-06-18-zh3r0-2020-Hastads-Message.html)

Abstract

CTF writeup: RSA repeats itself but forgets to introduce e in zh3r0 CTF 2020's Hastad's Message. Try exponents with CRT and integer roots for the broadcast attack.

Hastad's message

Description

My friend Hastad send me a message but I am not able to read it. Only thing I know is its different each time.

```
nc crypto.zh3r0.ml 7419
```

Author : Finch

Lets see what happens on netcatting the server

```
nc crypto.zh3r0.ml 7419
```

N:

```
23960949631136549010580709371447459320771722990641656469289715710142295495124948621158
68784708320832152950267853909006264659819488151364488491710721381266236620835811231235
24934214911084876467798479135553652324453231005868105543373968023216875838535640143229
73591145624358708717154959936810785468753949008206142023804817260040790518084021622384
35386692073568639229890343210781884297661625657347662211184231307242754693826193284542
12784796014673369330012680763470784402635271214889409550964162782791787987966909879494
46654662309459137968716222579092928758172393534109727678426842059547321144586394122734
488337726519323
```

CT:

```
22705918735106609862578354197108939246763445931879070926885855919243404749948030941257
90838213003345918442362706090099733614759297585031370534079477199926780672133456917308
84130984948269760409377387182408382901865063563197855973716061186698464514471396611589
44456549041342029102797654906767131709085534561472066009144741135274315448354388291160
98258112374635331610330152253489038186540922667451031193236349754454836321363492850673
03046354417457399725704445620542851629021518600614724115418587100535103614591024058009
10086788687364544808589567255632810731714686014811542832677962673594407069777753280116
5480737197887
```

```
nc crypto.zh3r0.ml 7419
```

N:

```
19791835797467968357781023201751418927414446960804616978761511157078113663179187861424
08433043929875201642682751172975811601770709181072177811510642108983609144633566487367
91159860244995533680186181809806753789666722702772230043458044560794905096445003155276
68853314227967157724434593288928717409769329504108075955720491368982617136637949677075
90480625585697326316968347122508822711187583489368489077330825469074935036149904683987
67823114213212935815968919570639968229448807484139129184494951877238538469903987740134
11469435139373416719302794535340647770663325255798177030402073444060330933741585572502
```

```
434974728609629
```

CT:

```
10711359293091554845881300822684711074568687131225620493949233624467865101717568049146
34894090857282586789210229391708624669355959025339364211156874967849670891344853449971
99917903705965734942982845512088539263240274820894572553642496792937455455963900961424
87991844667316810725319734547236480559387130970727056695739296995521658873612344594309
07475094510164028964628026368007770468736220153983649313051226993386499337203803136521
93728768205093848432184171198617228268176928190507827420519780759559546448919723446746
84686471352127887490340543050726681324525005791961696941767799080026081048866749630207
206154339441796
```

Seems like its encrypting the same message with different N and a same e each time which we dont know

This could be a variant of Hastad's broadcast attac as the name of the challenge suggests. One can read about RSA attacks and their implementations on ENOENT's Blog

If we have a small e, we know each $M \pmod{N_i}$

So if we can get e messages somehow, we know

$$M^e < N_1 N_2 N_3 \dots N_e$$

and

$$M^e = C$$

There would be no wrap-around of modulus $N_1 N_2 \dots N_e$. We can simply calculate the eth root of M^e to get M

But we dont know what the e is!

No problems, we can request the server as many times we wish, Basic idea is to apply Chinese Remainder Theorem over the set of C's and N's till we get a valid decryption.

```
from pwn import remote
from sympy.ntheory.modular import crt
from gmpy2 import iroot

HOST, PORT = "crypto.zh3r0.ml", 7419
n = []
c = []

REM = remote(HOST, PORT)
n.append(int(REM.recvline().strip()[2:]))
c.append(int(REM.recvline().strip()[3:]))
REM.close()
e=2
while True:
    print(e)
    REM = remote(HOST, PORT)
    n.append(int(REM.recvline().strip()[2:]))
    c.append(int(REM.recvline().strip()[3:]))
    resultant, mod = crt(n,c)
    value, is_perfect = iroot(resultant,e)
```

```
if is_perfect:
    print(bytes.fromhex(hex(value)[2:]).decode())
    break
e+=1
REM.close()
```

Its great that you have come till here. As promised here is your flag:
zh3r0{RSA_1s_0n3_of_th3_b4st_encrypt10n_Bu66y}

zh3r0-RSA`1s`0n3`of`th3`b4st`encrypt10n`Bu66y”

Interactive controls are omitted from this PDF. See the canonical HTML version.